数据结构与算法分析-4

人工智能与自动化学院 谭毅华

yhtan@hust.edu.cn 87556301

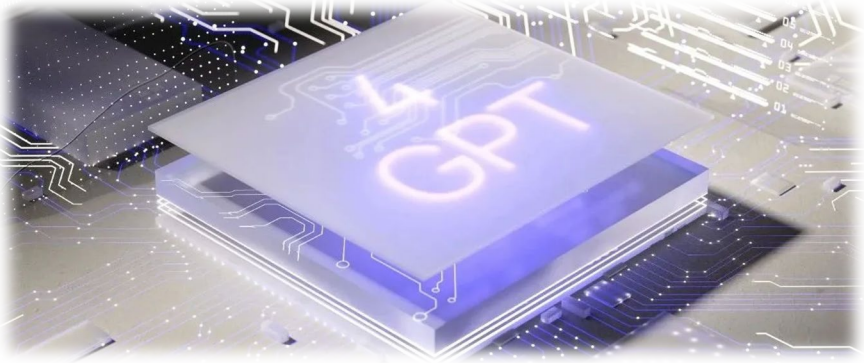
上节课回顾

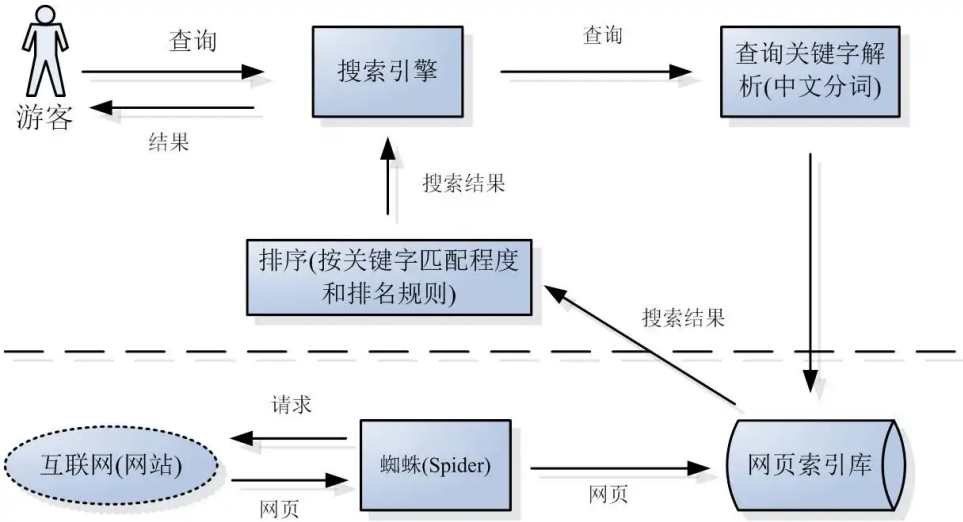
- 两种受限的线性结构
  - 栈：先进后出,在表尾操作
  - 队列：后进先出，队首删除，队尾插入
- 基本操作
  - 栈：Push，Pop，GetTop
  - 队列：EnQueue, DeQueue
- 递归
  - 利用栈实现子过程的逐层调用
  - 递归有三个条件：可终止返回，调用自身，可分解为简单的相似过程
- 例子：四则运算，汉诺塔

華中科技大學 人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

4. 串

- 4.1串类型的定义
- 4.2串的表示和实现
- 4.3串的模式匹配算法





4.1 串的定义

- 串(String)，是由零个或多个字符组成的有限序列，也称字符串。一般记为：

  $s = 'a_1a_2a_3\ldots a_n'$

  其中s称为串名，是用单引号括起来的字符序列

  C语言中字符串是用双引号括起来的字符序列

  a_i ($1 \leqslant i \leqslant n$)是字符，可以是字母、数字或其它符号

  n是串s的长度。

- 零个字符的串称为空串null ->C语言NULL

- 串中任意个连续的字符组成的子序列称为该串的子串。包含子串的串相应地称为主串。通常称字符在序列中的序号为该字符在串中的位置。子串在主串中的位置则以子串的第一个字符在主串中的位置来表示。

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

串的例子

- 称两个串是相等的，当且仅当这两个串的值相等。也就是说，只有当两个串的长度相等，并且各个对应位置的字符都相等时才相等。

  a = ‘WU’,　　b = ‘HAN’

  c = ‘WUHAN’　　d = ‘WU HAN’

- 值得一提的是，串值必须用一对单引号括起来，但单引号本身不属于串，它的作用只是为了避免与变量名或数的常量混淆而已。

華中科技大學 人工智能与自动化学院

串的抽象数据类型

ADT String {

数据对象：$D=\{a_i \mid a_i \in CharacterSet,\ i=1,2,\cdots,n,\ n\geqslant 0\}$

数据关系：$R_1=\{<a_{i-1},a_i> \mid a_{i-1},a_i\in D,\ i=2,\cdots,n\}$

基本操作：

StrAssign (&T, chars)

初始条件：chars 是字符串常量。

操作结果：生成一个其值等于 chars 的串 T。

StrCopy (&T, S)

初始条件：串 S 存在。

操作结果：由串 S 复制得串 T。

StrEmpty (S)

初始条件：串 S 存在。

操作结果：若 S 为空串，则返回 TRUE，否则返回 FALSE。

StrCompare (S, T)

初始条件：串 S 和 T 存在。

操作结果：若 S > T，则返回值 > 0；
若 S = T，则返回值 = 0；
若 S < T，则返回值 < 0。

StrLength (S)

初始条件：串 S 存在。

操作结果：返回 S 的元素个数，称为串的长度。

ClearString (&S)

初始条件：串 S 存在。

操作结果：将 S 清为空串。

Concat (&T, S1, S2)

初始条件：串 S1 和 S2 存在。

操作结果：用 T 返回由 S1 和 S2 联接而成的新串。

SubString (&Sub, S, pos, len)

初始条件：串 S 存在，$1\leqslant pos\leqslant StrLength(S)$
且 $0\leqslant len\leqslant StrLength(S)-pos+1$。

操作结果：用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。

Index (S, T, pos)

初始条件：串 S 和 T 存在，T 是非空串，
$1\leqslant pos\leqslant StrLength(S)$。

操作结果：若主串 S 中存在和串 T 值相同的子串，则返回它在主串 S 中第 pos 个字符之后第一次出现的位置；否则函数值为 0。

Replace (&S, T, V)

初始条件：串 S，T 和 V 存在，T 是非空串。

操作结果：用 V 替换主串 S 中出现的所有与 T 相等的不重叠的子串。

StrInsert (&S, pos, T)

初始条件：串 S 和 T 存在，$1\leqslant pos\leqslant StrLength(S)+1$。

操作结果：在串 S 的第 pos 个字符之前插入串 T。

StrDelete (&S, pos, len)

初始条件：串 S 存在，$1\leqslant pos\leqslant StrLength(S)-len+1$。

操作结果：从串 S 中删除第 pos 个字符起长度为 len 的子串。

DestroyString (&S)

初始条件：串 S 存在。

操作结果：串 S 被销毁。

}ADT String

華中科技大學 人工智能与自动化学院
A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

串的基本操作

- 最小操作子集：StrAssign、StrCompare、StrLength、Concat、SubString。基于此，可以进行子串查找定位。

```
int Index(String S, String T, int pos) {
  if(pos > 0) {
      n = StrLength(S); m = StrLength(T); i = pos;
    while(i <= n – m + 1) {
      SubString(sub, S, i, m);
      if(StrCompare(sub, T) != 0) ++i;
      else return i;
     } //while
  } // if
  return 0;
} //Index
```


華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

4. 2串的表示和实现

- 定长顺序存储表示

```
#define MAXSTRLEN 255;
typedef unsigned char SSTRING[MAXSTRLEN+1];
```


華中科技大學 人工智能与自动化学院

字符数组的连接

```
Status Concat(SString &T, SString S1, SString S2) {
    if(S1[0]+S2[0] <= MAXSTRLEN) {
      T[1..S1[0]] = S1[1..S1[0]];
      T[S1[0]+1..S1[0]+S2[0]] = S2[1..S2[0]];
      T[0] = S1[0] + S2[0];
    }
    else if(S1[0] < MAXSTRLEN) {     //截断
      …
    }
    …
}
```


華中科技大學 人工智能与自动化学院

串的堆分配存储表示

- 堆分配存储表示

```
Typedef struct {
    char *ch;   //若是非空串，则按串长分配存储区，否则ch为NULL
    int length; //串长度
} Hstring;
```


華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

连接操作

```
Status Concat(Hstring &T, HString S1, HString S2) {
    //S1与S2合并至T
   if(T.ch) free(T.ch);  //释放旧空间
    if(!(T.ch = (char )malloc((S1.length+S2.length)*sizeof(char))))
        exit(OVERFLOW);
   T.ch[0..S1.length-1] = S1.ch[0..S1.length-1];//赋S1的值
    T.length = S1.length + S2.length;
    T.ch[S1.length..T.length-1] = S2.ch[0..S2.length-1];
    return OK;
}
```


華中科技大學 人工智能与自动化学院

串的插入操作

```
Status StrInsert(Hstring &S, int pos, Hstring T) {//在S的pos前插入子串T
  if(pos<1||pos>S.length+1) return ERROR; //pos是否合法
  if(T.length) {//T非空，则重新为S分配空间，插入T
      if(!(S.ch = (char *)realloc(S.ch, S.length+T.length)*sizeof(char))))
       exit(OVERFLOW);
    for(i=S.length-1; i>=pos-1; --i) //先右移pos后的字符串
       S.ch[i+T.length] = S.ch[i];
    S.ch[pos-1..T.length-2] = T.ch[0..T.length-1];
    S.length += T.length;
  }
  return OK;
}
```


華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

串的块链存储表示

```
#define CHUNKSIZE 80
typedef struct Chunk {
    char ch[CHUNKSIZE];
    struct Chunk *next;
} Chunk;
typedef struct {
    Chunk *head, *tail;
    int curlen;
}LString;
```

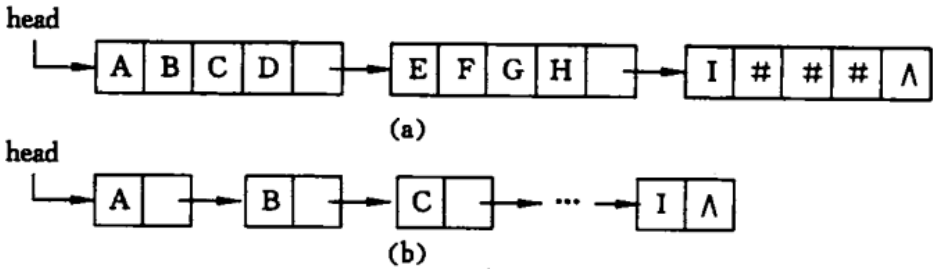


图 4.2　串值的链表存储方式

(a) 结点大小为 4 的链表；(b) 结点大小为 1 的链表

$$存储密度 = \frac{串值所占的存储位}{实际分配的存储位}$$

華中科技大学 人工智能与自动化学院

4.3 串的模式匹配算法

- 串的模式匹配：子串的定位操作。
- 子串称为模式串，主串称为目标。
- 定长顺序存储结构的匹配算法

```
int Index(SString S, SString T, int pos) {
    i = pos; j = 1;
    while(i<=S[0] && j<=T[0]) {
        if(S[i]==T[j]) {++i; ++j;}
        else {i = i – j + 2;  j = 1;}
    }
    if(j>T[0]) return i-T[0];
    else return 0;
}
```

例：　目标S: Huazhong University
　　模式T: ver
　　匹配结果：13

華中科技大學 人工智能与自动化学院

朴素的模式匹配(B-F算法)

第1趟

```
                ↓ i=3
S   a b b a b a
T   a b a
            ↑ j=3
```

第2趟

```
              ↓ i=2
S   a b b a b a
T     a b a
        ↑ j=1
```

第3趟

```
                ↓ i=3
S   a b b a b a
T       a b a
          ↑ j=1
```

第4趟

```
                        ↓ i=6
S   a b b a b a
T         a b a
                      ↑ j=4
```

计算复杂度：

最坏情况： T为m，S为n，需要(n-m+1)*m次比较，大多m << n，故复杂度为O(n*m)

低效原因： 每趟重新比较时，目标串的检测指针要回溯

華中科技大學 人工智能与自动化学院

改进的模式匹配

- D.E.Knuth、J.H.Morris和V.R.Pratt同时提出来的，故称为KMP算法
- 核心思想：消除每趟失配后的指针回溯，即可提高模式匹配效率

指针回溯：i=i-j+2, j=1

第1趟

```
                i=3
S   a b b a b a
T   a b a
            j=3
```

第2趟

```
              i=2
S   a b b a b a
T     a b a
        j=1
```

指针回溯：i=3, j=1

第3趟

```
                i=3
S   a b b a b a
T       a b a
          j=1
```


華中科技大學 人工智能与自动化学院

带来的新问题

- 若i不回溯，j应该退到何处？
- 给定模式串，用next[j]表示新的位置,如何求？

- **next[j]与主串S无关,只与模式串T有关？为什么？**

```
S  a b b a b d d e k l
T      a b d d k
         a b d d k        ×
           a b d d k      ×
             a b d d k    ×
               a b d d k
```

此时未知，比较a与e

華中科技大學 人工智能与自动化学院

运用KMP算法的匹配过程

第1趟 目标 a c a b a a b a a b c a c a a b c

模式 a b a a b c a c　　(p_1 p_2 p_3 p_4 p_5 p_6 p_7 p_8)

× j=2 ⇨ next(2) = 1，下次p_1

第2趟 目标 a c a b a a b a a b c a c a a b c

模式 a b a a b c a c

× j=1(特殊) ⇨ 下次p_1, 目标指针进 1

第3趟 目标 a c a b a a b a a b c a c a a b c

模式 a b a a b c a c

× j=6 ⇨ next(6) = 3,

下次p_3

第4趟 目标 a c a b a a b a a b c a c a a b c

模式 (a b) a a b c a c √

華中科技大學 人工智能与自动化学院

改进原理

$$
\begin{array}{lcccccccccccc}
S & s_1 & \cdots & s_{i-j} & s_{i-j+1} & s_{i-j+2} & s_{i-j+3} & \cdots & s_{i-1} & s_i & s_{i+1} & \cdots & s_{n-1} \\
 & & & & \| & \| & \| & & \| & \times & & & \\
T & & & & p_1 & p_2 & p_3 & \cdots & p_{j-1} & p_j & \cdots & p_m &
\end{array}
$$

则有 $s_{i-j+1}\, s_{i-j+2}\, s_{i-j+3} \cdots s_{i-1} = p_1\, p_2 \cdots p_{j-1}$ (1)

按照依次回溯法：下一趟，为使模式T与目标S匹配，必须满足

$$p_1\, p_2 \cdots p_{j-1} \cdots p_m = s_{i-j+2}\, s_{i-j+3} \cdots s_{i-j+2+m}$$

如果 $p_1 \cdots p_{j-2} \neq p_2 \cdots p_{j-1}$ (2)

则立刻可以断定

$p_1 \cdots p_{j-2} \neq s_{i-j+2}\, s_{i-j+3} \cdots s_{i-1}$ 下一趟必不匹配

華中科技大學 人工智能与自动化学院

改进原理–续

$$
\begin{array}{lcccccccccccc}
S & s_1 & \cdots & s_{i-j} & s_{i-j+1} & s_{i-j+2} & s_{i-j+3} & \cdots & s_{i-1} & s_i & s_{i+1} & \cdots & s_{n-1} \\
 & & & \| & \| & \| & \| & & & \times & & & \\
T & & & p_1 & p_2 & p_3 & \cdots & & p_{j-1} & p_j & \cdots & p_m &
\end{array}
$$

同样的道理：如果 $p_1 \cdots p_{j-3} \neq p_3\, p_4 \cdots p_{j-1}$ 下一趟必不匹配

直到某个k, $p_1 \cdots p_k \neq p_{j-k}\, p_{j-k} \cdots p_{j-1}$

但， $p_1 \cdots p_{k-1} == p_{j-k+1}\, p_{j-k+1} \cdots p_{j-1}$

故有： $p_1 \cdots p_{k-1} == s_{i-k+1}\, s_{i-k+1} \cdots s_{i-1}$

$$
\begin{array}{ccccc}
 & \| & \| & \| & \\
 & p_{j-k+1} & p_{j-k+1} & \cdots & p_{j-1}
\end{array}
$$

下一趟可以直接用 p_k 与 s_i 继续比较。

$$\Downarrow$$

$$p_1 \cdots p_{k-1} == p_{j-k+1}\quad p_{j-k+1}\quad \cdots\quad p_{j-1}$$

仅和模式串有关，如何确定k?

華中科技大學 人工智能与自动化学院

K的确定

- 当第j个字符失配时，不妨令 k = next[j]表示
- 它仅依赖于模式T 本身前 j 个字符的构成，与目标串无关.设模式$T = p_1 \cdots p_{m-1} p_m$

$$
\mathbf{next(j)} = \begin{cases} 0, & j = 1 \quad \text{子串保持不变，主串下移1位} \quad (1) \\ k, & 1 < k < j \text{且使得} p_1 \cdots p_{k-1} = p_{j-k+1} p_{j-k} \cdots p_{j-1} \text{的最大整数} \quad (2) \\ 1, & \text{其它情况} \quad (3) \end{cases}
$$

j	1	2	3	4	5	6	7	8
T	a	b	a	a	b	c	a	c
next(j)	0	1	1	2	2	3	1	2

華中科技大學 人工智能与自动化学院

KMP算法实现代码

```
int Index_KMP(SString S, SString T, int pos) {
    // 利用模式串 T 的 next 函数求 T 在主串 S 中第 pos 个字符之后的位置的
    //   KMP 算法。其中,T 非空,1≤pos≤StrLength(S)。
    i = pos;    j = 1;
    while (i <= S[0] && j <= T[0]) {
        if ( j = = 0 || S[i] = = T[j]) { ++i;   ++j; }        // 继续比较后继字符
        else  j = next[j];                                    // 模式串向右移动
    }
    if (j > T[0])  return  i - T[0];                          // 匹配成功
    else return 0;
} // Index_KMP
```

算法 4.6

条件里为什么使用j==0?

華中科技大學 人工智能与自动化学院

$p_1 \quad p_2 \quad p_3 \cdots p_{j-k+1} \cdots \quad p_{j-1} \quad p_j \quad p_{j+1} \cdots p_m$

$p_1 \quad \cdots \quad p_{k-1} \cdots \quad p_x$

p_k

求next[j+1]

$$next(j)=\begin{cases}0, & j=1 \quad \text{子串保持不变，主串下移1位} \quad (1)\\ k, & 1<k<j\text{且使得}p_1\cdots p_{k-1}=p_{j-k+1}p_{j-k}\cdots p_{j-1}\text{的最大整数} \quad (2)\\ 1, & \text{其它情况} \quad (3)\end{cases}$$

若已知next[j]=k,则 $P_1 \cdots P_{k-1} = P_{j-k+1} \cdots P_{j-1}$ 且k为满足上式最大的值，因此j处失配时只需比较P_k与P_j的值：

（1）若相等，显然与P_{j+1}比较的应为P_{k+1}

（2）若不等，相当于字串在k处失配，下一步转到next[k]与P_j再次比较

- 递推思想：即已知next[1]=0，…，next[*j*]=*k*，求next[*j*+1]。

① 判断模式串的p_j是否等于p_k。

② 若两者相等，有next[*j*+1]=*k*+1；

否则由于模式串在p_k处失配，应将模式串的next[*k*]与p_j再做比较（相当于子串右滑），即取*k*=next[*k*]，转①。

華中科技大學 人工智能与自动化学院

get_next函数实现

```
void getnext(Sstring T, int &next[])
{// 求模式串T的next函数值并存入数组next
        i=1; j=0;
        next[1]=0;
        while (i<T[0])
        {
                if (j==0||T[i]==T[j])
                  { i++;j++; next[i]=j;}
                else j=next[j];
        }
}
```

算法复杂度为O(m)

華中科技大學 人工智能与自动化学院 A-ambitious 积极进取 U-unique 锐意创新 T-tolerant 包容并蓄 O-open 开放交流

Take-home Message

- 串的基本概念和基本操作算法
- 串的简单模式匹配算法和KMP算法
- KMP算法利用了模式串内在的相似性简化指针移动